PROJECT REPORT

ON

“CampusConnect: Student Academic & Communication Portal”

INDIVIDUAL MODULE REPORT

Announcements Management Module

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted by:

ANJALI KUMARI

University Roll No.: 24EARCS020

Department of Computer Science & Engineering

Arya College of Engineering & I.T.

SP-42, RIICO Industrial Area, Kukas, Jaipur (Rajasthan)

Project Guide: Er. Ram Babu Buri

Head of Department: Dr. Akhil Pandey

Academic Session: 2026–27

CERTIFICATE

This is to certify that the individual project report entitled “CampusConnect: Student Academic & Communication Portal — Announcements Management Module” has been prepared by Anjali Kumari (24EARCS020), a student of B.Tech Computer Science & Engineering at Arya College of Engineering & I.T., Jaipur, under the academic guidance of Er. Ram Babu Buri during the session 2026–27.

The report describes the assigned announcements-related contribution within the CampusConnect group project. The report is intended for academic evaluation. Implementation details are described at the level supported by project notes; any item marked conceptual, proposed, or requiring verification must be checked against the final source code and running application before formal certification.

Project Guide	**Head of Department**
Er. Ram Babu Buri	Dr. Akhil Pandey

Department of Computer Science & Engineering

Arya College of Engineering & I.T., Jaipur

DECLARATION

I, Anjali Kumari (24EARCS020), hereby declare that this report titled “CampusConnect — Announcements Management Module” has been prepared for academic submission as an individual account of my assigned module in the group project.

The report explains the purpose, requirements, design, expected workflow, interfaces, testing approach, limitations, and future improvements for the announcements section. It does not claim that every proposed behaviour has been deployed or verified. Any implementation-specific details and test outcomes must be validated against the final branch and application before submission.

I understand that the group project is shared work and that this report focuses only on the announcements-related module assigned to me.

Place: Jaipur, Rajasthan

Name: Anjali Kumari

Roll No.: 24EARCS020

ACKNOWLEDGEMENT

I express my sincere gratitude to Er. Ram Babu Buri, Project Guide, for academic guidance and support during the CampusConnect project. I am thankful to Dr. Akhil Pandey, Head of the Department, and the Department of Computer Science & Engineering at Arya College of Engineering & I.T., Jaipur, for providing an environment to learn and apply software engineering concepts.

I also thank the CampusConnect team for coordinating the frontend, backend, database, and integration work. The announcements module depends on the larger portal architecture, role definitions, authentication, and shared API conventions, so collaboration and communication are important to its design.

Finally, I acknowledge the documentation and development tools used to plan, implement, review, and test the module. This report distinguishes the intended design from features whose implementation status must be verified in the final repository.

ABSTRACT

CampusConnect is a student academic and communication portal intended to bring college-related information and workflows into one place. The announcements section is a key communication feature: it provides a central place for authorized users to publish institutional notices and for students and teachers to view relevant updates.

This individual report focuses on the announcements management module assigned to Anjali Kumari (24EARCS020). It describes the motivation for a centralized notice feed, user roles, functional and non-functional requirements, conceptual architecture, user workflows, data model, API interaction, validation, error handling, and a practical test plan.

The wider project uses a React-based frontend, a Java Spring Boot backend, REST-style communication, and a MySQL database. The conceptual flow presented here is React UI → API client → backend controller/service → repository/data layer → database. The exact endpoint names, table names, permissions, and publication states must be confirmed against the current implementation. Real-time WebSocket notifications are treated as a future enhancement unless verified in the repository.

The report is structured as a software engineering document rather than a claim of production deployment. It includes diagrams to communicate intended behaviour and a traceability matrix to connect requirements with validation activities.

TABLE OF CONTENTS

1. Introduction
2. Project Background and Scope
3. Technologies and Tools
4. Stakeholders and User Roles
5. Requirement Analysis
6. Functional Requirements
7. Non-Functional Requirements
8. System Architecture
9. UML and Design Diagrams
10. Data Model and Validation
11. Announcement Workflows
12. Interface and API Design
13. Security and Access Control
14. Error Handling and Reliability
15. Testing Strategy and Test Cases
16. Implementation Notes and Verification Status
17. Limitations and Risks
18. Future Enhancements
19. Conclusion
Appendix A: Requirement Traceability
Appendix B: Glossary
References

Note: page numbers may change when the editable Word document is modified.

1. INTRODUCTION

1.1 Purpose

The purpose of the announcements module is to make important college updates accessible through a structured portal instead of relying only on scattered messages, notice boards, or informal group chats. It should help authorized publishers create clear notices and help users find the latest relevant information.

1.2 Problem Statement

Students may miss academic deadlines, event updates, timetable changes, examination notices, or department-level instructions when information is distributed across several channels. A centralized announcements section can improve visibility, consistency, and discoverability.

1.3 Objectives

- Provide a single portal section for viewing current announcements.
- Allow authorized users to create and maintain notices, subject to role permissions.
- Present each notice with clear metadata such as title, content, publication date, author, and audience where supported.
- Validate user input and provide understandable success or error feedback.
- Integrate the announcements UI with the shared CampusConnect frontend and backend architecture.
- Keep the module maintainable so future categories, search, pinning, and notification features can be added.

1.4 Intended Audience

The report is intended for the project guide, faculty evaluators, team members, and developers who need to understand the announcements module's purpose, design, and verification needs.

2. PROJECT BACKGROUND AND SCOPE

2.1 CampusConnect Overview

CampusConnect is a college-focused academic and communication portal developed as a group project. The larger concept includes role-aware student and teacher access, dashboards, profiles, announcements, assignments, chat, and club-related information. This individual report is limited to the announcements section.

2.2 In-Scope Capabilities

- Viewing a list of announcements and opening a notice for details.
- Displaying title, body, and available metadata in a readable layout.
- Providing a publisher workflow for creating or maintaining notices where implemented.
- Validating required fields and showing clear feedback.
- Connecting the frontend to backend endpoints through the project API client.
- Persisting announcement data through the project data layer where the backend implementation supports it.

2.3 Out-of-Scope / Not Assumed

- Guaranteed push notifications, email/SMS delivery, or WebSocket-based live updates.
- Advanced audience targeting unless the schema and UI implement it.
- Analytics, read receipts, and acknowledgement tracking unless verified.
- Claims of production deployment, measured performance, or complete security certification.

2.4 Assumptions and Dependencies

The module depends on the shared frontend routing, API base configuration, backend availability, user identity/role model, database connectivity, and agreed project conventions. If authentication or authorization middleware is incomplete in the current branch, publisher controls must not be described as fully protected until verified.

3. TECHNOLOGIES AND TOOLS

Technology / Tool	Role in the Project	Module Relevance
React + JavaScript	Frontend UI	Renders announcements list, detail and publisher form views.
Vite	Frontend development/build tooling	Runs and builds the React application.
Tailwind CSS	UI styling	Supports responsive layouts and reusable visual styles.
Axios	HTTP client	Sends requests from the frontend to REST endpoints.
Java + Spring Boot	Backend platform	Hosts REST controllers and business logic in the wider project.
Spring Data JPA / Hibernate	Persistence layer (project stack)	Maps backend entities to relational data where configured.
MySQL	Relational database	Stores persistent user and announcement records according to actual schema.
Git and GitHub	Version control and collaboration	Supports the announcements branch, reviews and team integration.
VS Code / browser developer tools	Development and debugging	Used to edit code and inspect requests, responses and UI behaviour.

3.1 Technology Boundaries

The table describes the known project stack and the intended role of each component. It is not proof that every listed technology is used by every announcements feature. The exact implementation should be confirmed in the current branch, particularly the entity mapping, API routes, and authorization checks.

4. STAKEHOLDERS AND USER ROLES

Actor	Typical Need	Announcements Interaction
Student	Find current academic and campus updates.	View notices and inspect details; filtering only if implemented.
Teacher / Staff Publisher	Communicate course or campus information.	Create or update notices if the role is authorized by the application.
Portal Administrator	Maintain reliable official information.	May manage or moderate notices if an administrator role exists.
Project Developer	Maintain module behaviour and integration.	Implement UI, API calls, validation, persistence and error handling.
Faculty Evaluator	Assess scope, quality and correctness.	Review requirements, diagrams, implementation evidence and tests.

4.1 Role and Permission Principle

Viewing announcements may be available to authenticated portal users, while create/update/publish actions should be restricted to authorized roles. This is a design requirement; it must be enforced on the server rather than only hiding buttons in the frontend. Actual role names and rules should match CampusConnect's implemented security model.

5. REQUIREMENT ANALYSIS

5.1 Functional Requirements

ID	Requirement
ANN-FR-01	The system shall display a list of available announcements to permitted users.
ANN-FR-02	Each list item shall show a title and concise content preview where available.
ANN-FR-03	The system shall allow a user to open an announcement and read its full content.
ANN-FR-04	An authorized publisher shall be able to submit a new announcement when creation is implemented.
ANN-FR-05	The form shall validate required values such as title and content before submission.
ANN-FR-06	The system shall show a success or failure message after a save/publish request.
ANN-FR-07	The system shall load announcement data from the backend rather than relying solely on hard-coded UI data.
ANN-FR-08	The system shall handle an empty list and API failure without crashing the page.
ANN-FR-09	Update, delete, publish/unpublish, audience filtering and search shall be supported only if included in the approved implementation.

5.2 Business Rules

- An announcement should have a meaningful title and non-empty content.
- The publisher identity should be associated with the record when the system supports author tracking.
- Only authorized users may create, modify, or publish official notices.
- Dates and status values must be consistent; expired or unpublished notices should follow the application’s defined visibility rules.
- The frontend must not report a successful save unless the backend response indicates success.

6. FUNCTIONAL REQUIREMENTS — DETAIL

6.1 View Announcements

When the user opens the announcements section, the interface should request the current list from the backend, show a loading state while waiting, render available notices, and provide an empty-state message if no records are returned. A failed request should result in a safe error message and, where appropriate, a retry option.

6.2 View Announcement Details

Selecting a notice should display its complete content and available metadata. The page should preserve readable line breaks, avoid overflowing long content, and clearly distinguish the title, body, author, and date fields when present.

6.3 Create / Publish Announcement

An authorized publisher should be able to enter a title, body, and any additional fields defined by the actual application. Before sending the request, the frontend should validate required values. The backend must repeat critical validation and enforce authorization. After the response, the UI should display a truthful result and refresh the list or detail view.

6.4 Update / Delete

Update and delete operations are optional capabilities unless verified in the current code. If supported, the system should verify that the current user is allowed to change the selected notice, confirm destructive actions where appropriate, and handle stale or missing records.

6.5 Search, Categories and Audience

Search, categories, priority labels, department targeting, and audience-specific visibility are useful extensions. They should be described as implemented only if matching controls, backend filters, and schema fields exist.

7. NON-FUNCTIONAL REQUIREMENTS

Category	Requirement / Quality Goal	Verification Approach
Usability	Notices should be easy to scan and readable.	Review labels, spacing, contrast and empty states.
Responsiveness	The interface should remain usable on common desktop and mobile widths.	Test multiple viewport sizes.
Reliability	API errors should not crash the announcements page.	Simulate unavailable backend and non-success responses.
Security	Publisher actions must be authorized on the server.	Test unauthenticated and unauthorized requests.
Data integrity	Required fields should not be persisted as invalid empty values.	Submit invalid data through UI and API.
Maintainability	UI, API interaction and backend persistence should remain separated.	Inspect module boundaries and code organisation.
Performance	List retrieval should be appropriate for the expected project dataset.	Measure response time only in a defined test environment.
Compatibility	The UI should work in the supported browser environment.	Run basic cross-browser smoke checks if required.

No numeric performance target is claimed here because no measured baseline has been supplied. Any target should be agreed with the project guide and measured under documented conditions.

8. SYSTEM ARCHITECTURE

The conceptual architecture separates the presentation layer, HTTP/API communication, backend request handling, business rules, and persistence. The frontend presents announcements and sends requests. The backend validates input and permissions, executes business logic, and accesses persistent records through its data layer.

- Presentation layer: React components for list, detail and publisher form views.
- API client: Axios or the shared project client for request configuration and response handling.
- Controller layer: maps HTTP requests to backend operations, using actual route names from source code.
- Service layer: applies business rules, validation and authorization decisions where implemented.
- Repository/data layer: reads and writes records using the configured persistence approach.
- Database: stores announcement records and related author/role data according to the real schema.

This is a logical architecture diagram, not a claim that every component shown is already implemented with these exact class names.

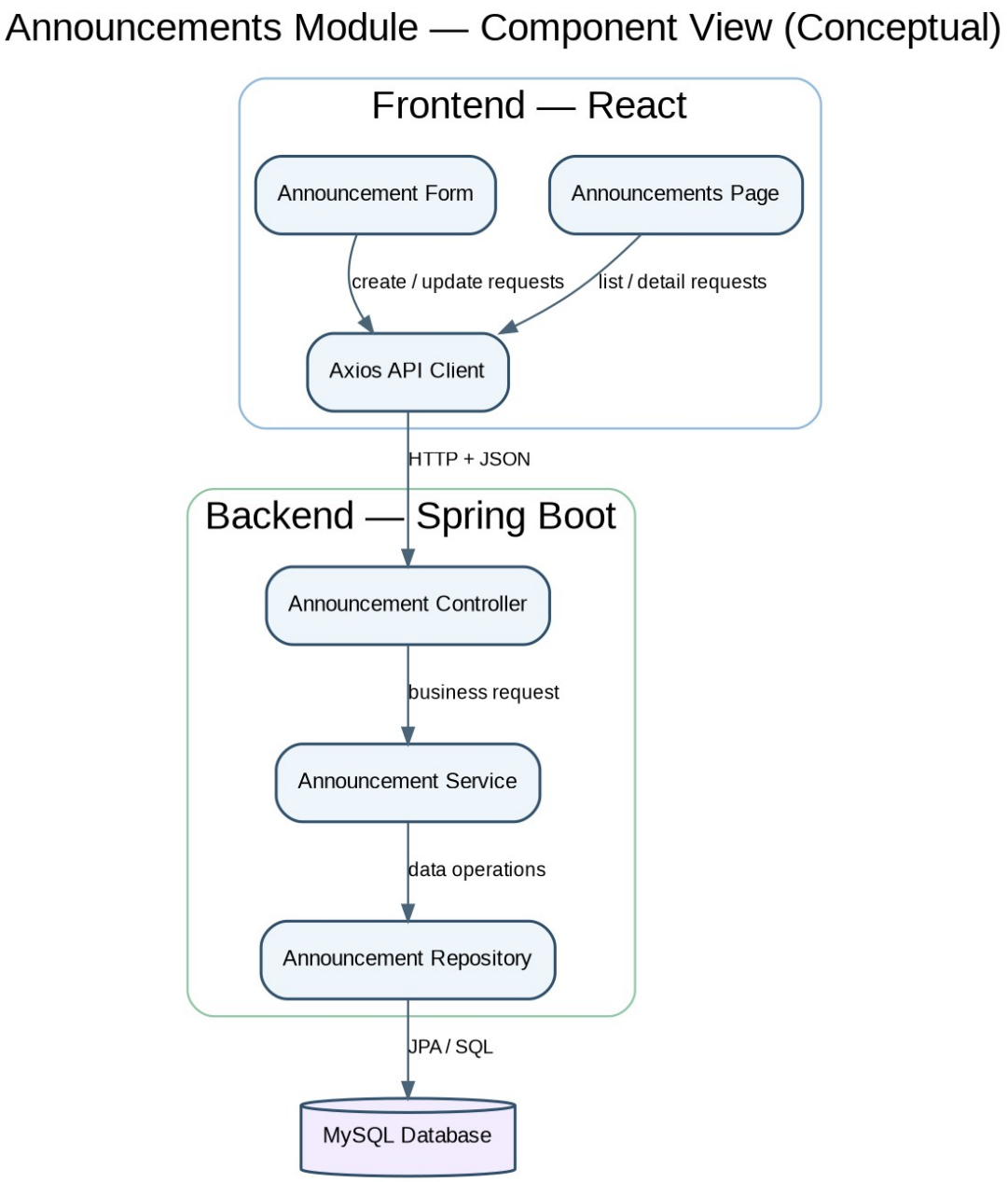


Figure 1. Conceptual component architecture for the announcements module.

9. UML AND DESIGN DIAGRAMS

9.1 Use Case Diagram

The use-case view distinguishes readers from authorized publishers. Students and teachers may view notices; only permitted roles should create or manage official notices. The diagram is conceptual and should be adjusted if the project defines a different publisher role.

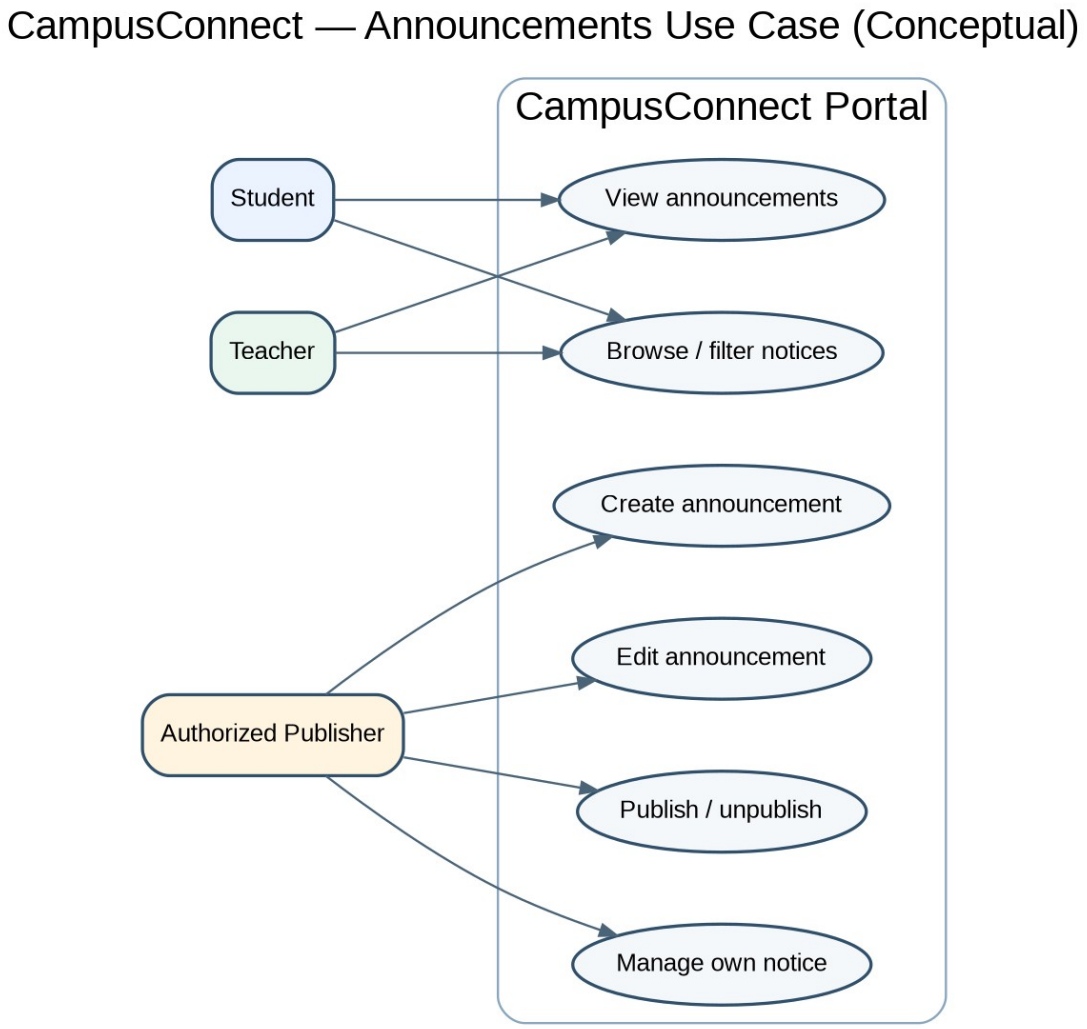


Figure 2. Announcements use cases and actor interactions.

9.2 Activity Diagram

The activity diagram outlines a publisher entering data, validating required fields, submitting a request, and receiving success or error feedback. It highlights that both client-side and server-side validation are needed.

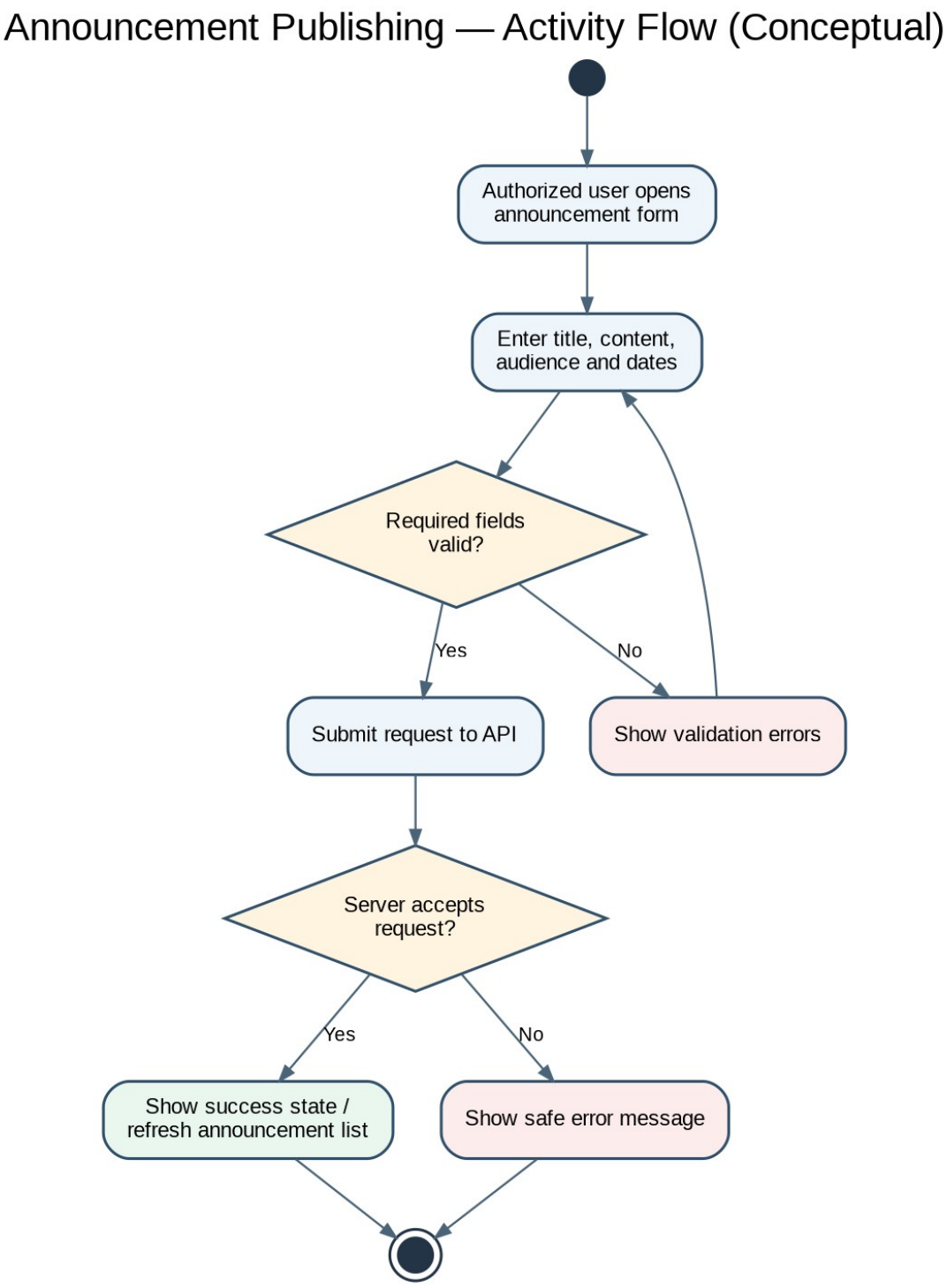


Figure 3. Activity flow for creating/publishing an announcement.

9.3 Sequence Diagram

The sequence diagram illustrates the logical request-response path from browser UI to React component, API client, backend controller/service, repository and database. The concrete URL and HTTP method must be taken from the current source code.

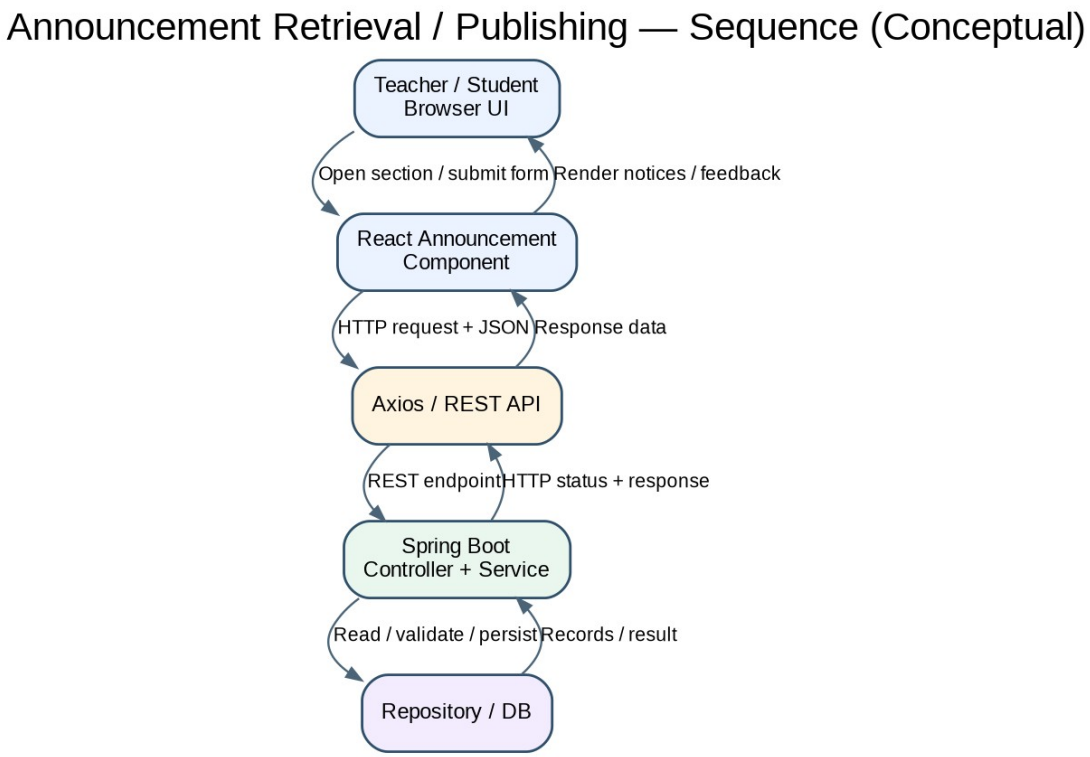


Figure 4. Conceptual request/response sequence.

9.4 Conceptual Data Model

The conceptual data model includes an announcement record and an author/user relationship. Field names are illustrative. They must be reconciled with the actual JPA entity and MySQL schema before the diagram is presented as an implemented database design.

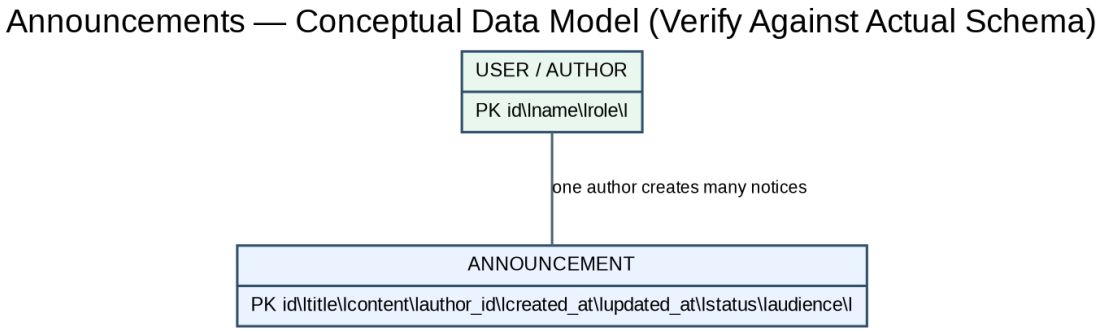


Figure 5. Conceptual announcement data model — verify against actual schema.

10. DATA MODEL AND VALIDATION

10.1 Suggested Announcement Attributes

Attribute	Purpose	Validation / Note
id	Unique identifier	Primary key; generated or assigned according to actual entity.
title	Short notice heading	Required; trim whitespace; enforce a sensible length.
content	Full notice body	Required; reject blank content; render safely.
author_id	Publisher reference	Foreign key/reference if author tracking is implemented.
created_at	Creation timestamp	Set consistently by backend or persistence layer.
updated_at	Last modification time	Updated when content changes, if supported.
status	Draft/published/archived state	Use only states that exist in the implementation.
audience	Target group or visibility	Optional; include only if schema and business rules support it.

10.2 Input Validation

- Trim title and content before checking emptiness.
- Reject requests missing required values with a clear validation response.
- Set maximum lengths consistent with database column definitions.
- Do not trust author IDs or role claims sent by the browser; derive identity from authenticated server context where available.
- Encode or safely render user-provided text to avoid script injection in the browser.
- Validate date ordering if start/end or expiry dates are supported.

10.3 Database Verification

Before final submission, inspect the actual entity class, repository, migration/schema file and MySQL table to confirm field names, nullability, data types, indexes and relationships. This report intentionally avoids claiming that illustrative fields exactly match the live database.

11. ANNOUNCEMENT WORKFLOWS

11.1 Reader Workflow

1. User signs in to CampusConnect according to the shared authentication flow.
2. User opens the announcements section from the dashboard/navigation.
3. Frontend requests the available announcement records.
4. The interface displays loading, populated, empty, or error state as appropriate.
5. User opens a notice to read the complete content and metadata.

11.2 Publisher Workflow

6. Authorized publisher opens the create/edit form.
7. Publisher enters the notice title, body and any supported metadata.
8. Frontend validates required fields and prevents clearly invalid submission.
9. API client sends the request to the configured backend endpoint.
10. Backend validates the request, checks permission, and performs the persistence operation if valid.
11. Frontend interprets the response, displays accurate feedback and refreshes the relevant view.

11.3 Failure Workflow

If the server is unavailable, the request times out, validation fails, or a permission check rejects the operation, the UI should preserve the user’s entered values where safe, show a non-sensitive explanation, and avoid claiming that the notice was saved.

12. INTERFACE AND API DESIGN

12.1 Frontend Views

View / Component (conceptual)	Purpose	Key UI States
Announcements list	Browse notices	Loading, populated, empty, error
Announcement detail	Read full notice	Loaded, missing record, error
Create/edit form	Enter or update notice	Idle, validation error, submitting, success, failure
Shared navigation/dashboard entry	Open module	Visible according to the app's navigation/role model

12.2 API Contract Guidance

The following is a contract template, not a claim about exact implemented route names. Replace placeholders with the routes and DTOs found in the final source code.

Operation	Illustrative HTTP Pattern	Expected Behaviour
List notices	GET /api/announcements (example only)	Return permitted announcement records.
Get one notice	GET /api/announcements/{id} (example only)	Return one notice or a not-found response.
Create notice	POST /api/announcements (example only)	Validate, authorize and create a record.
Update notice	PUT/PATCH /api/announcements/{id} (example only)	Update only if supported and authorized.
Delete/archive	DELETE or status update (example only)	Perform only if included in approved functionality.

12.3 Response Handling

Frontend code should distinguish successful responses from validation errors, unauthorized/forbidden responses, not-found responses, and server failures. User-facing messages should be helpful without exposing stack traces, database details, or secrets.

13. SECURITY AND ACCESS CONTROL

13.1 Authorization

Announcement creation and editing are sensitive because they affect official information. Permission checks must be enforced in the backend. Hiding a create button in React is a usability measure, not a security boundary.

13.2 Input and Output Safety

- Validate all inputs on the server, even if the frontend validates them too.
- Render notice content as text or sanitize rich content using an approved mechanism.
- Use parameterized persistence operations through the project's ORM/data layer.
- Return only the fields needed by the client.
- Keep database credentials and signing secrets outside committed source files.
- Use HTTPS in deployed environments and avoid logging passwords, tokens or sensitive user data.

13.3 Auditability

For a future-ready portal, the module may record author and timestamps for notice creation/update. Administrative audit history, approvals, and deletion logs are enhancements unless already present.

14. ERROR HANDLING AND RELIABILITY

Scenario	Expected Behaviour	Evidence to Capture
No announcements available	Display a clear empty state.	Screenshot or UI test result.
Backend unavailable	Show an error state; avoid a blank/crashed page.	Network error and rendered message.
Blank title/content	Reject submission and identify missing fields.	Form validation test.
Unauthorized publisher	Reject create/update request server-side.	HTTP response and UI handling.
Unknown announcement ID	Show not-found state or navigate back safely.	API/UI test.
Database persistence failure	Return controlled error; do not show false success.	Server log plus client response.
Long or special-character content	Render without layout break or script execution.	Browser test with representative content.

The table defines expected behaviour. It does not mean every scenario has already passed. Record actual results after executing tests against the current integrated branch.

15. TESTING STRATEGY AND TEST CASES

Testing should cover frontend states, backend validation, role permissions, persistence, and integration. The team should record the test environment, build/commit identifier, input data, expected result, actual result, and pass/fail status.

Test ID	Test Scenario	Expected Result	Status
ANN-T01	Open announcements page with records present	List renders available records.	To be executed / verified
ANN-T02	Open page with no records	Empty state appears.	To be executed / verified
ANN-T03	Submit blank title	Validation rejects request.	To be executed / verified
ANN-T04	Submit blank content	Validation rejects request.	To be executed / verified
ANN-T05	Valid authorized create request	Record is saved and success feedback is accurate.	To be executed / verified
ANN-T06	Create request from unauthorized user	Server denies operation.	To be executed / verified
ANN-T07	Backend returns 5xx / becomes unavailable	Safe error message; no false success.	To be executed / verified
ANN-T08	Open non-existent notice	Not-found state is handled.	To be executed / verified
ANN-T09	Render long text and punctuation	Content remains readable and safe.	To be executed / verified
ANN-T10	Check responsive layout	Controls remain usable at target widths.	To be executed / verified

15.1 Build and Integration Checks

- Run the frontend build and record the actual exit status.
- Run backend tests and record the test count/result from the current commit.
- Verify API calls in browser developer tools and inspect HTTP status codes.
- Check that created data remains available after refresh if persistence is implemented.
- Test the module after merging with the authentication and dashboard branches.

16. IMPLEMENTATION NOTES AND VERIFICATION STATUS

16.1 Contribution Focus

Anjali Kumari’s individual contribution is the announcements-related section of CampusConnect. The report therefore concentrates on notice browsing/publishing requirements, frontend-to-backend communication, validation, error states, and integration with shared portal navigation and role concepts.

16.2 Known Development Context

Project context records an `anjali-announcements` branch and earlier development/integration work. Notes from that work indicated backend tests had been reported as passing and the frontend build as clean at one point, while a publish API later returned an unexpected server error. These are historical development notes, not proof of the current branch status. Re-run tests and the publish workflow against the exact commit intended for submission.

16.3 Items to Verify Before Submission

- Confirm actual announcements controller/service/entity/repository names and routes.
- Confirm which operations are implemented: list, detail, create, update, delete, publish/unpublish.
- Confirm the database schema and whether author, status, audience and timestamp fields exist.
- Verify role-based access on the backend, not only in the UI.
- Run the publish operation and inspect the actual response to resolve any previously observed server error.
- Record real test outcomes and add screenshots from the running application.
- Do not describe WebSocket/live notifications as implemented unless source code and tests demonstrate them.

17. LIMITATIONS AND RISKS

- The module depends on the availability and correctness of shared authentication and API configuration.
- Exact schema and endpoint details may change as the group integrates parallel branches.
- Server-side publishing failures can prevent successful notice creation even when the frontend builds correctly.
- Without clear permission rules, unauthorized users could attempt to publish or modify notices.
- Without pagination or search, a very large notice feed may become harder to browse.
- Real-time delivery, email notifications, read receipts and audience targeting may require additional backend services and data structures.
- The diagrams in this report are conceptual until checked against final implementation.

17.1 Risk Mitigation

Use branch reviews and integration tests, define a stable API contract, validate requests at both client and server, keep permission checks in backend services/controllers, and capture reproducible evidence for any reported bug or successful test.

18. FUTURE ENHANCEMENTS

- Search by title/content and filter by category or date.
- Pin urgent notices and support expiry dates.
- Target notices by department, course, semester or role where policy permits.
- Add attachments such as circular PDFs with file validation and access controls.
- Add read receipts or acknowledgement for mandatory notices.
- Add in-app or email notifications after the basic publication flow is stable.
- Add scheduled publishing and archive/history view.
- Add audit trail for edits and administrative moderation.
- Add pagination and caching for large datasets.
- Add analytics for notice reach only if privacy and consent requirements are addressed.

These items are proposed future scope. They should not be represented as current capabilities unless verified in the repository.

19. CONCLUSION

The announcements module supports CampusConnect’s objective of centralizing academic and campus communication. A well-designed notice section helps users locate relevant updates and gives authorized publishers a consistent workflow for distributing information.

This report documents the module’s intended requirements, architecture, user interactions, conceptual data model, security considerations, error handling and test plan. It also records important verification points so that academic documentation remains aligned with the actual implementation. The next priority is to validate the publish workflow on the final integrated branch, confirm exact schema and route names, and attach genuine screenshots and test evidence.

The module can be extended with search, categories, expiry, targeted delivery and notifications after the core list/create flow and authorization behaviour are stable.

APPENDIX A: REQUIREMENT TRACEABILITY

Requirement	Design / Component	Suggested Test
ANN-FR-01 List notices	Announcements list + API client	ANN-T01
ANN-FR-02 Preview metadata	Announcement card/list item	ANN-T01
ANN-FR-03 Full details	Detail view + retrieval endpoint	ANN-T08
ANN-FR-04 Authorized create	Form + backend service	ANN-T05, ANN-T06
ANN-FR-05 Required validation	Frontend form + backend validation	ANN-T03, ANN-T04
ANN-FR-06 Accurate feedback	Response handling / UI state	ANN-T05, ANN-T07
ANN-FR-07 Backend data	API client + persistence layer	ANN-T01, ANN-T05
ANN-FR-08 Empty/error state	List component state handling	ANN-T02, ANN-T07

APPENDIX B: GLOSSARY

Term	Meaning
Announcement	A notice or update shared through the portal.
API	Application Programming Interface used by frontend and backend to exchange data.
REST	An architectural style commonly used for HTTP-based APIs.
Controller	Backend component that receives and routes HTTP requests.
Service	Backend layer for business rules and processing.
Repository	Data-access layer that queries or persists records.
CRUD	Create, Read, Update and Delete operations.
RBAC	Role-Based Access Control; permissions based on user role.
Validation	Checking that input satisfies required rules before processing.
Traceability	Linking each requirement to design elements and tests.

REFERENCES

- CampusConnect project repository: https://github.com/Adit841/CampusConnect
- React documentation: https://react.dev/
- Spring Boot documentation: https://docs.spring.io/spring-boot/
- Spring Data JPA documentation: https://docs.spring.io/spring-data/jpa/
- MySQL documentation: https://dev.mysql.com/doc/
- MDN Web Docs — HTTP and web platform references: https://developer.mozilla.org/
- Project source code, README, branch history, and test/build outputs should be consulted for implementation-specific details.